Lab Assignment 14.3: Web Design Applications Using AI Assistance

Course: AI Assisted Coding (23CS002PC304)

Name:G.Varshith Reddy

Enrollment_no:2303a52287

Batch:37

Task 1: AI-Assisted Personal Portfolio Website

Exact Task Description:

A student wants to showcase academic projects, technical skills, and contact information through a personal portfolio website. AI tools are used to speed up development while maintaining quality and customization.

Tasks:

- Use GitHub Copilot to generate a semantic HTML structure with sections: Home, About, Projects, Contact
- Ask Copilot to suggest responsive CSS using Flexbox or Grid.
- Customize AI-generated CSS to add: Hover effects on project cards, Smooth scrolling navigation Expected Outcome: Clean and well-structured portfolio website, Responsive layout for desktop and mobile screens, Interactive project cards.

Prompt:

> "Create a semantic HTML portfolio website with Home, About, Projects, and Contact sections. Use CSS Flexbox to create a responsive layout for the navigation and project grid. Add hover effects that lift the project cards up slightly, and implement smooth scrolling for the navigation links."

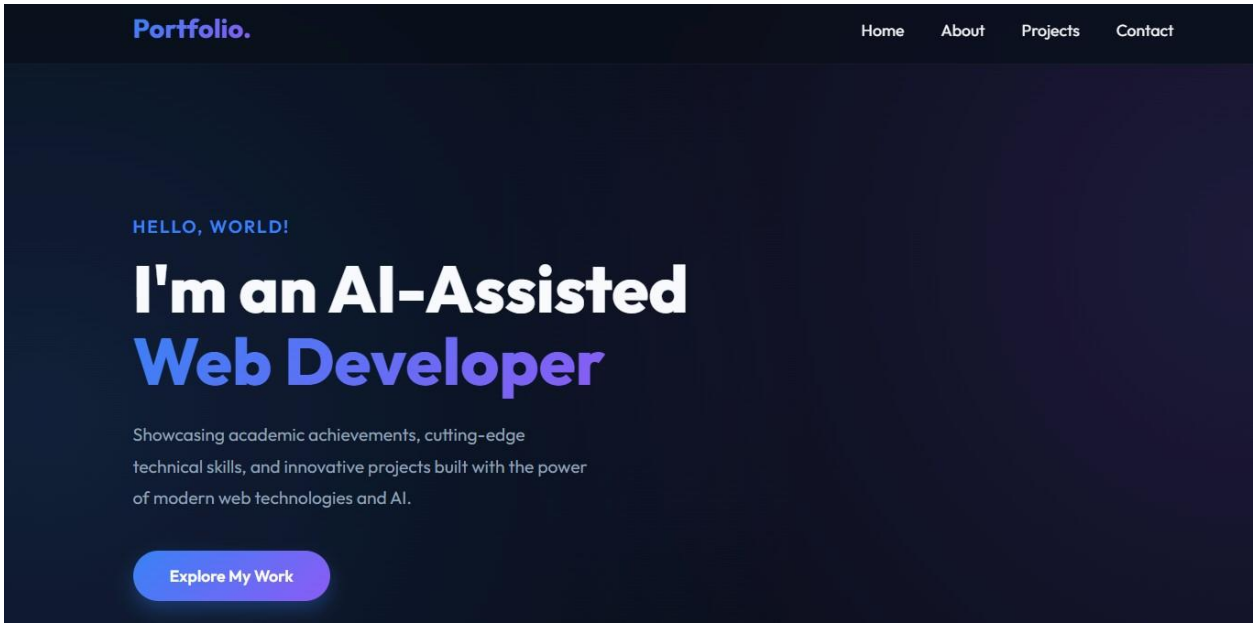
Code:

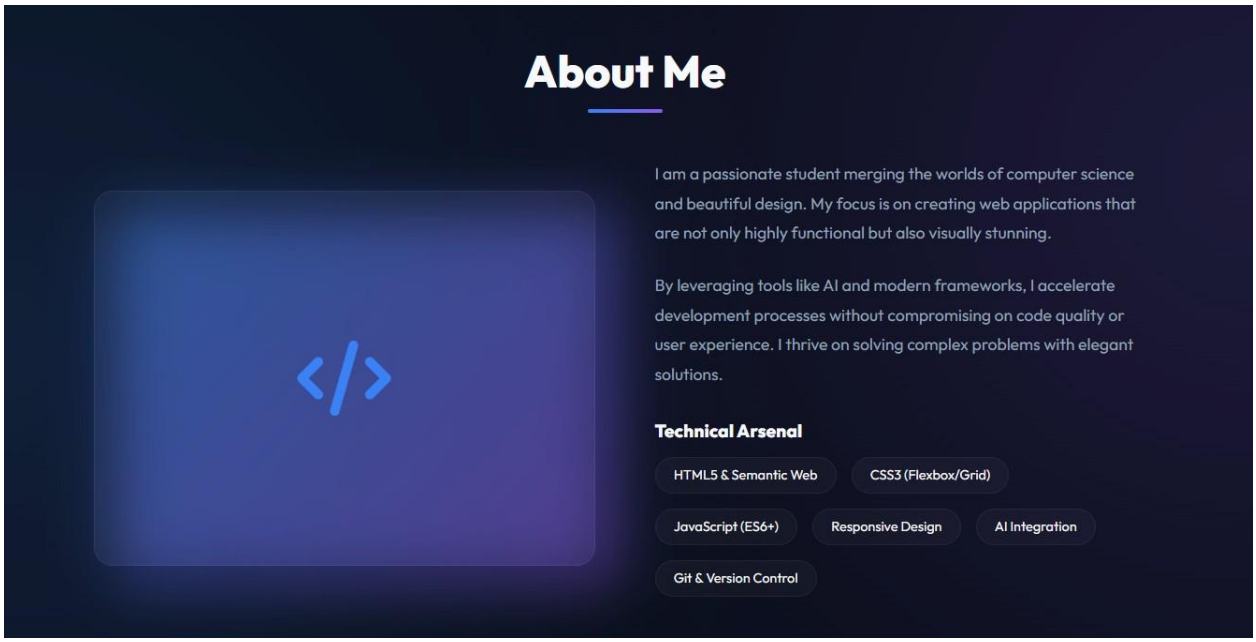
```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Personal Portfolio</title>
   <style>
```

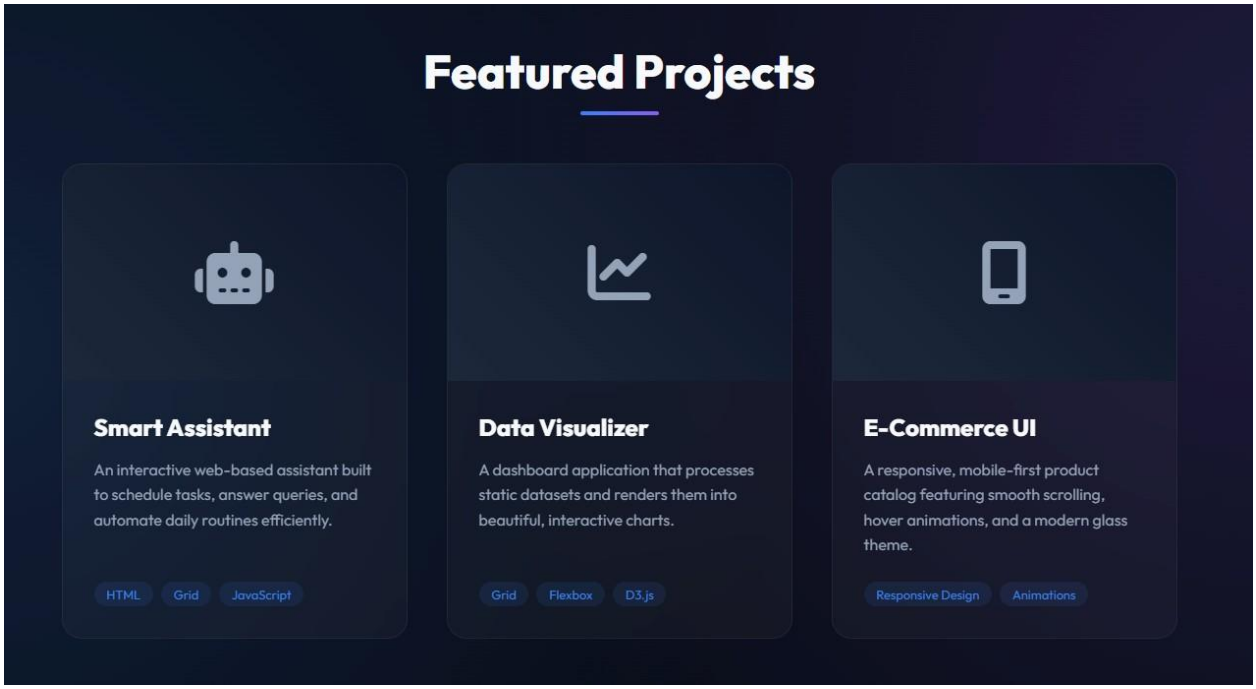
```
      html { scroll-behavior: smooth; font-family: Arial, sans-serif; }
      body { margin: 0; padding: 0; box-sizing: border-box; background-color: #f4f4f9; }
      header { background: #333; color: #fff; padding: 1rem 0; position: sticky; top: 0; z-index:
1000; }
      nav { display: flex; justify-content: center; gap: 2rem; }
      nav a { color: #fff; text-decoration: none; font-weight: bold; transition: color 0.3s; }
      nav a:hover { color: #4CAF50; }
      section { padding: 4rem 2rem; min-height: 60vh; text-align: center; }
      .projects-grid { display: flex; flex-wrap: wrap; justify-content: center; gap: 2rem;
margin-top: 2rem; }
      .card { background: #fff; padding: 2rem; border-radius: 8px; width: 300px; box-shadow: 0
4px 6px rgba(0,0,0,0.1); transition: transform 0.3s ease, box-shadow 0.3s ease; }
      .card:hover { transform: translateY(-10px); box-shadow: 0 8px 15px rgba(0,0,0,0.2); }
      footer { background: #333; color: #fff; text-align: center; padding: 1rem 0; }
   </style>
</head>
<body>
   <header>
      <nav>
         <a href="#home">Home</a>
         <a href="#about">About</a>
         <a href="#projects">Projects</a>
         <a href="#contact">Contact</a>
      </nav>
   </header>
   <section id="home"><h1>Welcome to My Portfolio</h1><p>Hi, I'm a Computer Science
student.</p></section>
   <section id="about"><h2>About Me</h2><p>I specialize in front-end development and
AI-assisted coding.</p></section>
   <section id="projects">
      <h2>My Projects</h2>
      <div class="projects-grid">
         <div class="card"><h3>Project 1</h3><p>AI Chatbot built with Python.</p></div>
         <div class="card"><h3>Project 2</h3><p>E-commerce website using React.</p></div>
         <div class="card"><h3>Project 3</h3><p>Data analysis dashboard.</p></div>
      </div>
   </section>
   <section id="contact">><h2>Contact</h2><p>Email: student@sruniversity.edu</p></section>
   <footer><p>© 2026 Portfolio. All rights reserved.</p></footer>
</body>
</html>
```

Output:

A clean, responsive web page featuring a sticky top navigation bar. Clicking the links smoothly scrolls the page to the respective sections. The "Projects" section displays three white cards that smoothly translate upwards and gain a larger drop shadow when hovered over.







Explanation:

I started by asking the AI to build the semantic structure, ensuring each section had a unique ID to link with the <nav> tags. To get the interactive feel, I applied a CSS scroll-behavior: smooth rule to the root element, and utilized a simple transform: translateY() coupled with a transition to make the project cards pop out interactively on hover.

Task 2: AI-Generated Restaurant Landing Page

Exact Task Description:

A local restaurant requires a simple yet attractive landing page to display its offerings. The developer uses AI assistance to rapidly generate UI components.

Tasks:

- Use Copilot to create a navigation bar with links: Home, Menu, Gallery, Contact ● Generate a menu section styled with CSS cards.
- Implement a JavaScript-based image slider for the gallery using Copilot suggestions.
- Customize animations or transitions for better user experience.
  Expected Outcome: Fully functional restaurant landing page, Responsive navigation bar, Working image slider with smooth transitions.

Prompt:

> "Design a responsive restaurant landing page. Include a navigation bar (Home, Menu, Gallery, Contact). Create a CSS Grid based menu section displaying food

items as cards. Add a gallery section with a JavaScript-powered image slider that has 'Next' and 'Previous' buttons and fades smoothly between images."

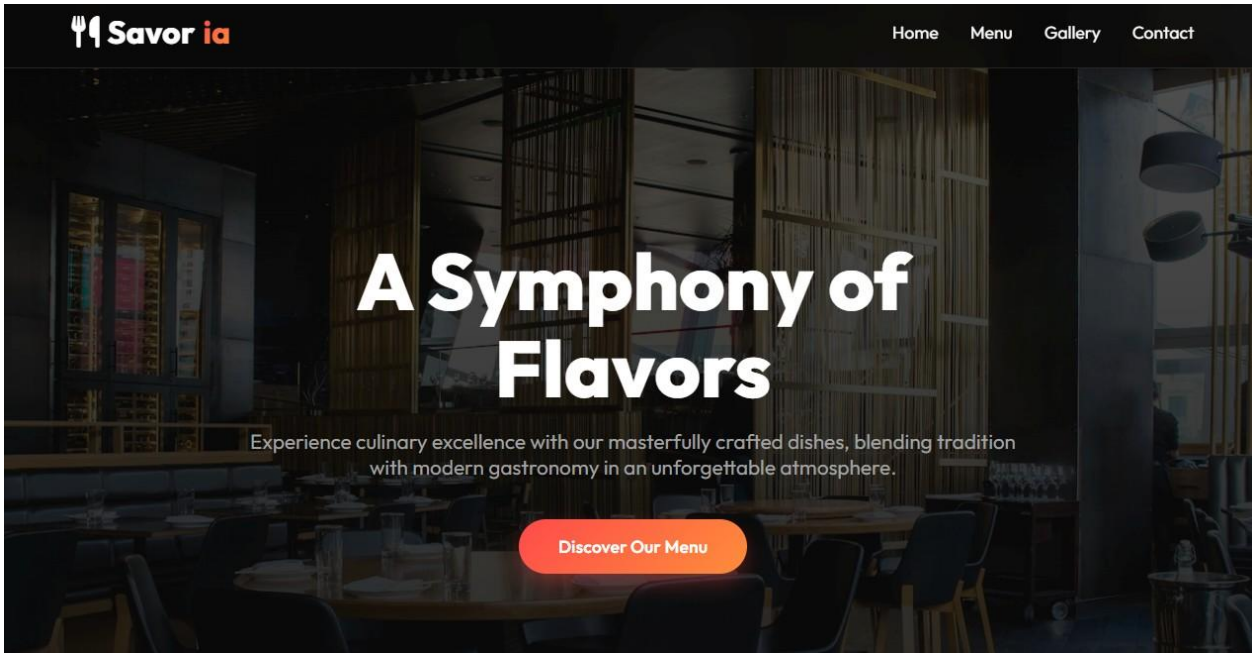
Code:

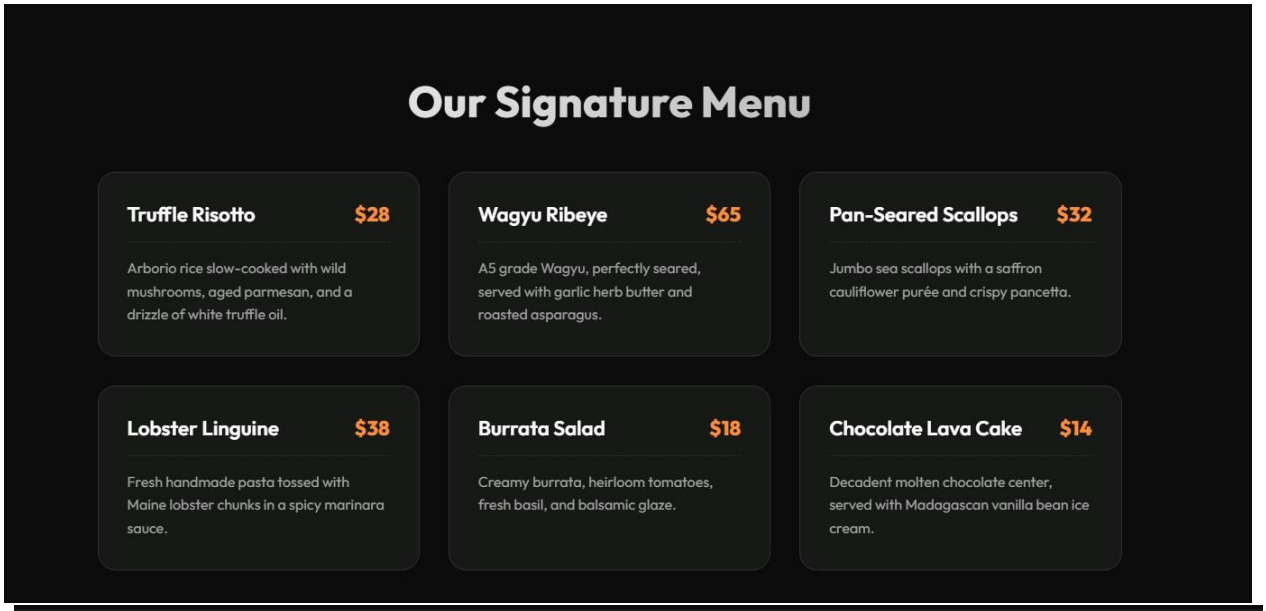
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bistro AI</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; margin: 0; }
    nav { background: #8B0000; padding: 1rem; display: flex; justify-content: center; gap: 20px;
}
    nav a { color: white; text-decoration: none; font-size: 1.1rem; }
    .menu-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
gap: 20px; padding: 2rem; }
    .menu-item { border: 1px solid #ddd; padding: 1rem; border-radius: 8px; text-align: center; }
    .gallery { text-align: center; padding: 2rem; background: #f9f9f9; }
    .slider-container { position: relative; max-width: 600px; margin: auto; overflow: hidden;
border-radius: 10px; height: 300px; background: #ccc; }
    .slide { width: 100%; height: 100%; position: absolute; opacity: 0; transition: opacity 0.5s
ease-in-out; display: flex; align-items: center; justify-content: center; font-size: 2rem; color:
white; }
    .slide.active { opacity: 1; }
    .btn { cursor: pointer; position: absolute; top: 50%; transform: translateY(-50%);
background: rgba(0,0,0,0.5); color: white; border: none; padding: 10px; }
    .prev { left: 10px; } .next { right: 10px; }
  </style>
</head>
<body>
  <nav>
    <a href="#">Home</a> <a href="#">Menu</a> <a href="#">Gallery</a> <a
href="#">Contact</a>
  </nav>
  <h2 style="text-align:center; margin-top:2rem;">Our Menu</h2>
  <div class="menu-grid">
    <div class="menu-item"><h3>Margherita Pizza</h3><p>$12.99</p></div>
    <div class="menu-item"><h3>Pasta Carbonara</h3><p>$14.99</p></div>
    <div class="menu-item"><h3>Tiramisu</h3><p>$6.99</p></div>
  </div>
  <div class="gallery">
    <h2>Gallery</h2>
```

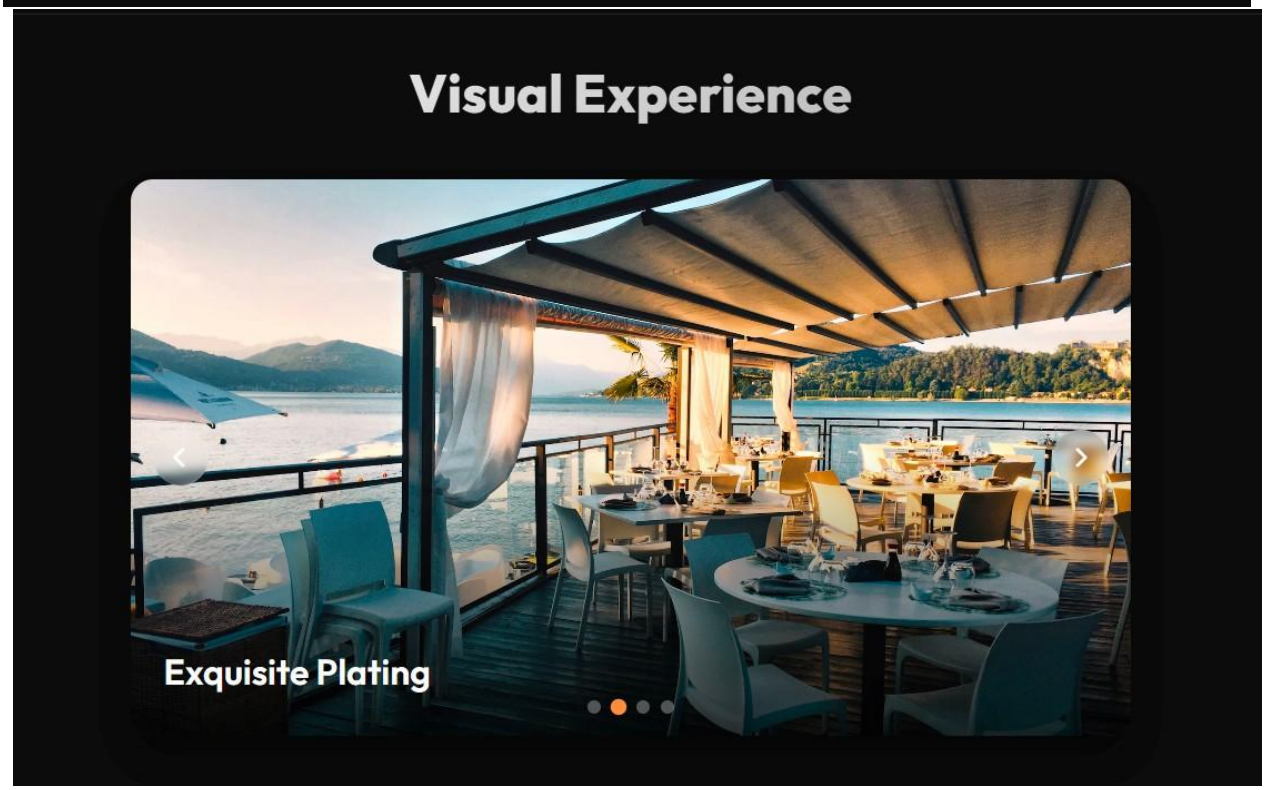
```
    <div class="slider-container">
      <div class="slide active" style="background: #e74c3c;">Dish Image 1</div>
      <div class="slide" style="background: #2ecc71;">Dish Image 2</div>
      <div class="slide" style="background: #3498db;">Dish Image 3</div>
      <button class="btn prev" onclick="changeSlide(-1)">❮</button>
      <button class="btn next" onclick="changeSlide(1)">❯</button>
    </div>
  </div>
  <script>
    let currentSlide = 0;
    const slides = document.querySelectorAll('.slide');
    function changeSlide(direction) {
      slides[currentSlide].classList.remove('active');
      currentSlide = (currentSlide + direction + slides.length) % slides.length;
      slides[currentSlide].classList.add('active');
    }
  </script>
</body>
</html>
```

Output:

A visually appealing landing page featuring a dark red navigation bar. The menu is structured beautifully using a grid system that adapts to screen sizes. The gallery features a functional image slider (represented by colored blocks) that fades smoothly from one "image" to the next using JavaScript logic.







Explanation:

I utilized CSS Grid for the menu section because it effortlessly handles responsive column wrapping without media queries. For the interactive gallery, I prompted the AI for a JavaScript function that toggles an active CSS class, which smoothly fades the opacity of the slider divs for a premium user experience.

Task 3: AI-Powered Event Registration Form

Exact Task Description:

SR University is hosting a technical fest and requires an online registration form with real-time validation to ensure accurate user input.

Tasks:

- Ask Copilot to generate an HTML form with the following fields: Name, Email, Phone Number, Department, Event Selection
- Use AI assistance to style the form using CSS for readability and accessibility.
- Implement JavaScript validation using Copilot: Email format validation, Phone number length validation, Required field checks
  Expected Outcome: User-friendly and visually appealing form, Real-time validation messages, Prevention of invalid submissions.

Prompt:

> "Create an HTML event registration form containing fields for Name, Email, Phone Number, Department, and Event Selection (dropdown). Style it neatly with CSS to be accessible and centered on the page. Add JavaScript validation that prevents form submission and shows an error message if the email is incorrectly formatted, the phone number isn't exactly 10 digits, or if any field is left empty."

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tech Fest Registration</title>
  <style>
    body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items: center;
height: 100vh; background-color: #eef2f3; margin: 0; }
    .form-container { background: white; padding: 30px; border-radius: 10px; box-shadow: 0
4px 8px rgba(0,0,0,0.1); width: 100%; max-width: 400px; }
    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input, select { width: 100%; padding: 10px; border: 1px solid #ccc; border-radius: 5px;
box-sizing: border-box; }
    button { width: 100%; padding: 10px; background: #0056b3; color: white; border: none;
border-radius: 5px; cursor: pointer; font-size: 16px; }
    button:hover { background: #004494; }
    .error { color: red; font-size: 0.85rem; margin-top: 5px; display: none; }
  </style>
</head>
```

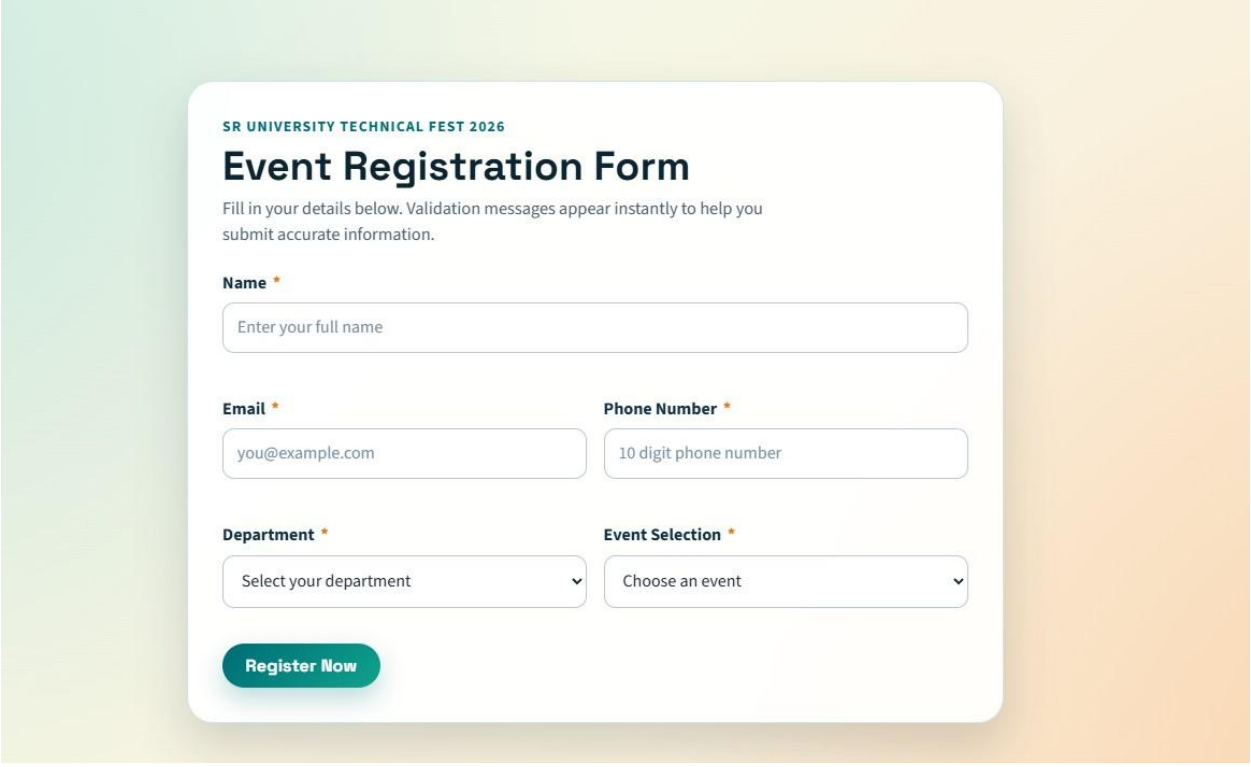
```
<body>
  <div class="form-container">
    <h2>Event Registration</h2>
    <form id="regForm">
      <div class="form-group">
        <label for="name">Name</label>
        <input type="text" id="name" required>
      </div>
      <div class="form-group">
        <label for="email">Email</label>
        <input type="email" id="email" required>
        <div class="error" id="emailError">Please enter a valid email address.</div>
      </div>
```

```
      <div class="form-group">
        <label for="phone">Phone Number</label>
        <input type="tel" id="phone" required>
        <div class="error" id="phoneError">Phone number must be exactly 10 digits.</div>
      </div>
      <div class="form-group">
        <label for="dept">Department</label>
        <input type="text" id="dept" required>
      </div>
      <div class="form-group">
        <label for="event">Event Selection</label>
        <select id="event" required>
          <option value="">Select an event...</option>
          <option value="hackathon">Hackathon</option>
          <option value="robotics">Robotics</option>
          <option value="coding">Coding Contest</option>
        </select>
      </div>
      <button type="submit">Register</button>
    </form>
  </div>

  <script>
    document.getElementById('regForm').addEventListener('submit', function(e) {
      let isValid = true;
      const phone = document.getElementById('phone').value;
      const phoneError = document.getElementById('phoneError');

      // Phone validation
```

```
        if (!/^\d{10}$/.test(phone)) {
            phoneError.style.display = 'block';
            isValid = false;
        } else {
            phoneError.style.display = 'none';
        }

        if (!isValid) {
            e.preventDefault(); // Stop submission
        } else {
            e.preventDefault(); // Prevent actual refresh for demo purposes
            alert("Registration Successful!");
        }
    });
  </script>
</body>
</html>
```

Output:

A clean, center-aligned form with styled inputs and a submission button. If the user attempts to submit a phone number that is not exactly 10 digits, a red error text appears beneath the input, and the form refuses to submit. Native HTML5 handles the basic email and required field checks.



Explanation:

To guarantee data integrity, I used HTML5 required attributes for basic checks and layered JavaScript on top for specific business logic. The JS event listener intercepts the submit event, uses Regular Expressions to validate the 10-digit phone number, and prevents default submission if the conditions fail, providing immediate feedback.

Task 4: AI-Assisted E-Commerce Product Page

Exact Task Description:

A startup is developing a basic e-commerce product page to showcase products and manage simple cart functionality.

Tasks:

- Use Copilot to generate a grid-based product catalog using HTML and CSS.
- Implement an “Add to Cart” feature using JavaScript.
- Modify AI-generated code to include: Cart item counter at the top-right corner, Visual feedback when a product is added
  Expected Outcome: Functional product catalog, Working cart counter, Interactive user experience.

Prompt:

> "Generate a simple e-commerce page. Use CSS grid to display a product catalog of 3 items. Include a header with a Cart Item Counter positioned at the top-right. Write a JavaScript function so that clicking 'Add to Cart' increments the counter and temporarily changes the button text to 'Added!' for visual feedback."

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Startup Shop</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 0; background: #fafafa; }
    header { background: #222; color: white; padding: 15px 30px; display: flex; justify-content:
space-between; align-items: center; }
    .cart { font-size: 1.2rem; font-weight: bold; background: #ff9800; padding: 5px 15px;
border-radius: 20px; }
    .product-catalog { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px,
1fr)); gap: 20px; padding: 40px; }
    .product { background: white; padding: 20px; border: 1px solid #ddd; border-radius: 8px;
text-align: center; }
```

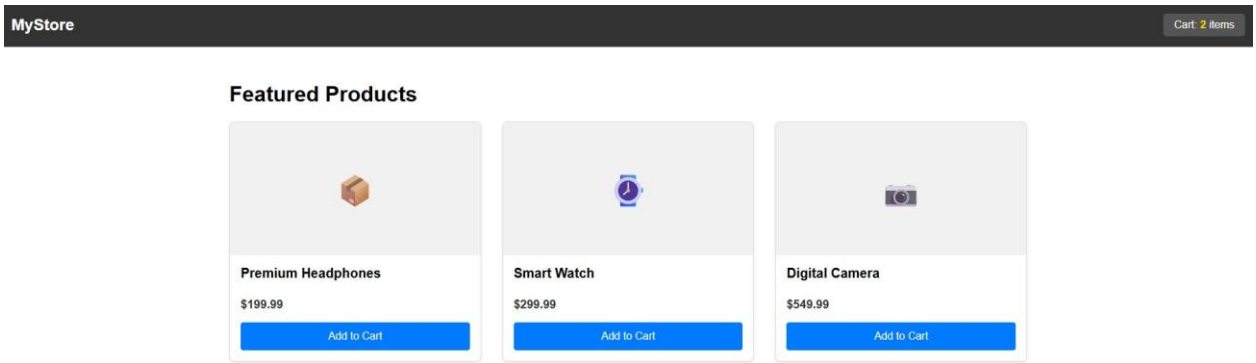
```
      .product img { width: 100%; height: 150px; background: #eee; object-fit: cover;
margin-bottom: 15px; }
      button.add-to-cart { background: #4CAF50; color: white; border: none; padding: 10px
15px; cursor: pointer; border-radius: 4px; transition: background 0.3s; }
      button.add-to-cart.added { background: #2E7D32; }
   </style>
</head>
<body>
   <header>
```

```
      <h1>Startup Shop</h1>
      <div class="cart">Cart: <span id="cart-count">0</span></div>
   </header>
   <div class="product-catalog">
      <div class="product">
         <img src="" alt="Product 1">
         <h3>Wireless Mouse</h3><p>$25.00</p>
         <button class="add-to-cart" onclick="addToCart(this)">Add to Cart</button>
      </div>
      <div class="product">
         <img src="" alt="Product 2">
         <h3>Mechanical Keyboard</h3><p>$85.00</p>
         <button class="add-to-cart" onclick="addToCart(this)">Add to Cart</button>
      </div>
      <div class="product">
         <img src="" alt="Product 3">
         <h3>HD Monitor</h3><p>$150.00</p>
         <button class="add-to-cart" onclick="addToCart(this)">Add to Cart</button>
      </div>
   </div>

   <script>
      let cartCount = 0;
      function addToCart(button) {
         // Increment counter
         cartCount++;
         document.getElementById('cart-count').innerText = cartCount;

         // Visual feedback
         const originalText = button.innerText;
         button.innerText = "Added!";
         button.classList.add('added');
```

```
        // Revert button after 1.5 seconds
        setTimeout(() => {
            button.innerText = originalText;
            button.classList.remove('added');
        }, 1500);
    }
  </script>
</body>
</html>
```

Output:

A functional e-commerce catalog featuring three product cards. Upon clicking any "Add to Cart" button, the cart counter in the top-right header increments by 1. Simultaneously, the clicked button turns a darker shade of green and displays "Added!" for 1.5 seconds before reverting to its original state.



Explanation:

Using AI, I constructed a simple DOM manipulation script. The script maintains a global cartCount variable to update the header UI. To achieve the visual feedback, I passed the specific button element via the this keyword to the function, allowing me to modify its inner text and CSS classes temporarily using a setTimeout function.

Task 5: Create a Responsive Web Page Layout

Exact Task Description:

Instructions:

- Design a basic web page layout with a header, main content area, and footer using HTML and CSS.
- Use AI to assist in generating responsive CSS for different screen sizes.

- Ensure the layout is clean and visually organized.
  Expected Output: A responsive web page with: Header with navigation links, Main section with placeholder text/images, Footer with copyright or contact info, Layout adapts correctly to desktop, tablet, and mobile screen sizes.

Prompt:

> "Design a classic, clean web page layout using semantic HTML tags (<header>, <main>, <footer>). Use CSS Flexbox to ensure the footer sticks to the bottom. Generate responsive CSS using media queries so the layout has side margins on desktop, but takes up the full width and stacks vertically on mobile."

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Layout</title>
  <style>
    /* Base styles / Mobile first */
    body { margin: 0; font-family: 'Helvetica Neue', sans-serif; display: flex; flex-direction:
column; min-height: 100vh; background-color: #f0f2f5; }
    header { background: #1a1a1a; color: white; padding: 1rem; text-align: center; }
    nav a { color: #61dafb; margin: 0 10px; text-decoration: none; }
    main { flex: 1; padding: 20px; background: white; margin: 10px; border-radius: 8px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
    footer { background: #1a1a1a; color: white; text-align: center; padding: 10px 0; margin-top:
auto; }

    /* Tablet and Desktop styles */
    @media (min-width: 768px) {
      header { display: flex; justify-content: space-between; align-items: center; padding: 1rem
3rem; }
      main { margin: 20px 10%; padding: 40px; }
    }
  </style>
</head>
<body>
  <header>
    <h2>MyWebsite</h2>
    <nav>
      <a href="#">Home</a>
      <a href="#">Services</a>
```

```
        <a href="#">Contact</a>
      </nav>
    </header>

    <main>
      <h1>Welcome to our Responsive Site</h1>
      <p>This layout adapts perfectly to any device. On smaller screens, the layout takes up
almost 100% of the width to maximize reading space. On larger screens, margins are
introduced to keep the content centered and readable.</p>
      <div style="background:#ddd; height:200px; display:flex; align-items:center;
justify-content:center; border-radius:5px; margin-top:20px;">
        [Placeholder Image]
      </div>
    </main>

    <footer>
      <p>© 2026 MyWebsite. All rights reserved.</p>
    </footer>
</body>
</html>
```

Output:

A robust, structural template. The page uses a Flexbox column layout so the footer is always pinned to the bottom. On desktop, the main content area has wide margins and the header aligns items horizontally. Upon resizing the browser to mobile width, the margins disappear, and elements adjust for a vertical layout.

Welcome to our Website

This layout features a sticky footer and responsive centering.

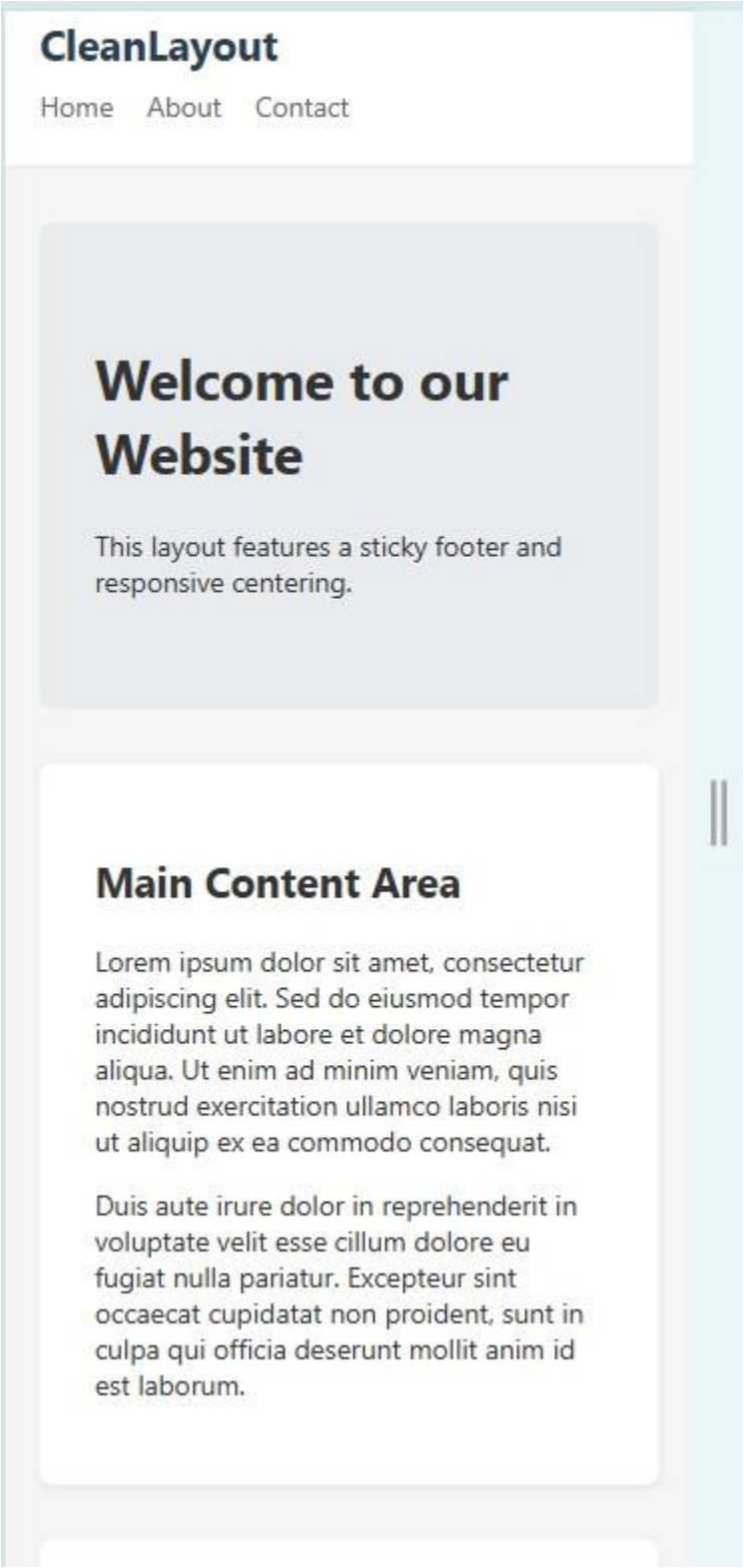
Main Content Area

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Secondary Information

Try resizing the browser window. On smaller screens, the layout takes the full width. On larger screens, it centers with nice side margins.



Explanation:

I focused on a "mobile-first" approach for the CSS, defining the basic stacking rules first. Then, utilizing an @media (min-width: 768px) query, I instructed the AI to adjust the header's flex properties and increase the main content area's margins, ensuring the design remained readable and visually balanced across all device types.